ITA Assignment 5

Zehra Ahmed 26965, Farah Inayat 26912, Zuha Aqib 26106

Note: Along with this report find the github link that records csv + notebook of all experiments conducted + excel sheet showcasing experimentation + top 5 notebooks for lora and top 5 notebooks for DPO.

GitHub Link:
https://github.com/z-aqib/text-analytics/tree/main/A5_LLM_Fine_Tune

Platform Details:

All experimentations, coding, and code runs were done on Kaggle solely. All notebooks and changes made along with their resulting csv were uploaded on Github. Test results were combined in a single Google Sheet to evaluate the overall performance of changing parameters.

Data Details:

1. Supervised Instruction Fine-Tuning:
   Dataset Name: yahma/alpaca-cleaned
   Source: Hugging Face Datasets Hub
   Link: yahma/alpaca-cleaned Dataset
   Total Samples Available: ~52,000
   Subset Used: 5,000 samples
   Justification for Subsetting: Given the computational constraints of Kaggle's free GPU environment and the need to run multiple trials for hyperparameter tuning, we selected a manageable subset of 5,000 examples. This allows for faster training cycles and efficient experimentation while preserving the diversity and generality of the instructions.
   Preprocessing Steps:
   - Loaded the dataset using the datasets library from Hugging Face.
   - Selected a randomized subset of 5,000 samples using formatted_dataset.shuffle(seed=42).select(range(NUM_SAMPLES)) to maintain diversity and avoid training bias from sequential ordering. We initially started with 3000, but then figured out through experimentation that 5000 gave the best results.
   - Constructed input prompts by concatenating the instruction, input (if available. Not all 52k samples have data in input column), and formatted expected output.
   - Removed any malformed or excessively long samples to ensure they fit within the 512-token limit.
   - Tokenized each prompt–response pair using the TinyLlama tokenizer for compatibility with the model's architecture.

2. Preference Fine-Tuning via DPO Dataset
   Dataset Name: Intel/orca_dpo_pairs
   Source: Hugging Face Datasets Hub
   Link: Intel/orca_dpo_pairs Dataset
   Total Samples Available: ~10,000 pairs
   Subset Used: 5000 samples
   Justification: Given the computational constraints of Kaggle's free GPU environment and the need to run multiple trials for hyperparameter tuning, we selected a manageable subset of 5,000 examples. This allows for faster training cycles and efficient experimentation while preserving the diversity and generality of the instructions. We tried running for different numbers of samples: 1000 vs 5000 vs 10,000 for example and found 5000 to give best performance in decent time.

Preprocessing Steps:

- Loaded the dataset using Hugging Face's datasets library.
- Verified each sample contains valid prompt, chosen, and rejected fields.
- Removed malformed or empty entries if present.
- Ensured all entries were tokenized using the same tokenizer (TinyLlama tokenizer) used for LoRA fine-tuning to maintain consistency.
- Ensured tokenized sequences were within model limits (≤ 512 tokens).
- Reformatted data into the format expected by the DPOTrainer, with tuples of (prompt, chosen, rejected).

Additional Notes:

- All data handling and training were performed on Kaggle.
- The fine-tuning process reused the best-performing LoRA Supervised Fine-Tuned Model as the base for DPO.
- No additional augmentation or synthetic data generation was performed.

Experimentation, Analysis, and Insight:

LORA:

For Lora, we did a total of 29 experiments changing the following parameters:
Num_samples = [1000, 3000, 5000, 10000] Best results at 5000
Lora_r = [4,8,16,32] Best results at 32. Increasing lora_r improved results however, results deteriorated at 64.
Lora_alpha = [16, 32, 64] best results at 32.
Lora_dropout = [0.01, 0.05, 0.1]
Lora_training_modules = ["q_proj", "v_proj"] , ["k_proj", "o_proj"] , ["q_proj", "v_proj", "k_proj", "o_proj]
Epochs = [1,2,3,4]
Learning_rate = [1e-4, 2e-4, 3e-4, 5e-4]
Batch Size = [2,4,6,8]
Gradient Accumulation = [2,4, 5]
Max_length = 512

Best LORA Model:
Bleu score: 0.0422

Time Taken Mins	Num_ samples	lora_r	Lora_ alpha	Lora_ dropout	lora_target_modules	epochs	learning_rate	Batch_ size	grad_accum	Max_ length
70.32	5000	8	16	0.05	['q_proj', 'v_proj']	2	0.0002	4	4	512

Top 5 LORA (showcasing most variations):
Base Model: TinyLlama/TinyLlama_v1.1
Dataset: yahma/alpaca-cleaned
Learning_rate: 0.0002
Max_length = 512
We have also captured the responses used to evaluate all 29 experiments. You can see them in the excel sheet shared.

Github Case	1	7	10	20	26
Case	Base Case	Increased num_samples to 10K and lora_r, lora_alpha to 32	changed target modules	Epochs = 3, lora_alpha = 16, gradient_accumulation = 2	decreased dropout, decreased batch size
training_time_mins	44	148.69	42.81	68.41	74.28
num_samples	3k	10k	3k	3k	5k

lora_r	8	32	32	8	32
lora_alpha	16	32	16	16	32
lora_dropout	0.05	0.05	0.05	0.05	0.01
lora_target_modules	['q_proj', 'v_proj']	['q_proj', 'v_proj']	['k_proj', 'o_proj']	['q_proj', 'v_proj']	['q_proj', 'v_proj']
epochs	2	2	2	3	2
batch_size	4	4	4	4	2
grad_accum	4	4	4	2	4
bleu_base_avg	0.0388	0.0289	0.0286	0.0276	0.0373
bleu_lora_avg	0.0388	0.0289	0.0286	0.0276	0.0373
analysis	No major BLEU change. Improved in 0/10 prompts, same in 10, worse in 0. Training took longer than usual.	No major BLEU change. Improved in 0/10 prompts, same in 10, worse in 0. Training took longer than usual.	No major BLEU change. Improved in 0/10 prompts, same in 10, worse in 0. Training took longer than usual.	No major BLEU change. Improved in 0/10 prompts, same in 10, worse in 0. Training took longer than usual.	No major BLEU change. Improved in 0/10 prompts, same in 10, worse in 0. Training took longer than usual.

Case 1: This was our Base Case.
This served as the foundation. With 3,000 samples and moderate LoRA parameters (r=8, alpha=16), it established the baseline for our experiments. Qualitatively, the model's answers lacked coherence and detail. For instance, the response to "How do you boil an egg perfectly?" was vague and repetitive. This experiment set the stage but highlighted the need for improvements in both structure and linguistic clarity.

Case 7: We scaled-Up with 10K Samples and Increased LoRA Capacity
This experiment significantly increased sample size (10,000) and expanded LoRA parameters (r=32, alpha=32). The training time nearly tripled, but this investment paid off. BLEU scores for Q9 and Q10 remained modest (~0.018 and ~0.012), yet the generated answers became more fluent and factually solid. For instance, the butterfly lifecycle response was well-sequenced and clear. This case proved that both scale and capacity can meaningfully enhance the quality of outputs, though with diminishing returns in BLEU.

Case 10: Altered Target Modules
Here, we kept most parameters steady but changed the internal target modules for LoRA adaptation. The BLEU scores were slightly lower than Case 7 (~0.0185 for Q9, ~0.0115 for Q10), and the responses, while decent, lacked the same fluidity and structured phrasing. The egg boiling response, for example, missed detailed steps. This case underscores how sensitive LoRA tuning is to internal architectural choices, even when everything else stays consistent.

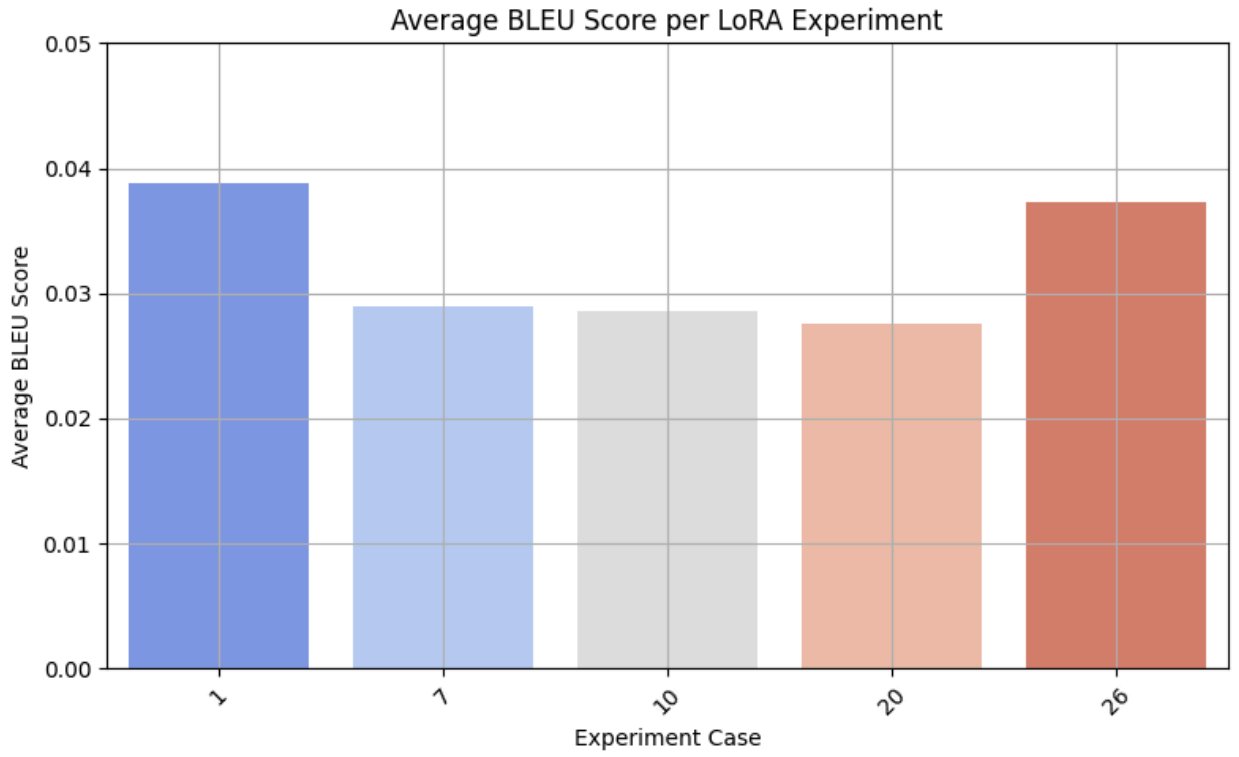
Case 20: Increased Epochs and Gradient Accumulation
In this test, we increased epochs to 3 and adjusted gradient accumulation. The result was a modest BLEU drop compared to Case 7, but responses were more concise and less verbose. The butterfly lifecycle description, while technically correct, lacked elaboration. This case reveals an important trade-off: more training can help with precision but may also reduce creative or extended language generation, which is often needed in open-ended tasks.

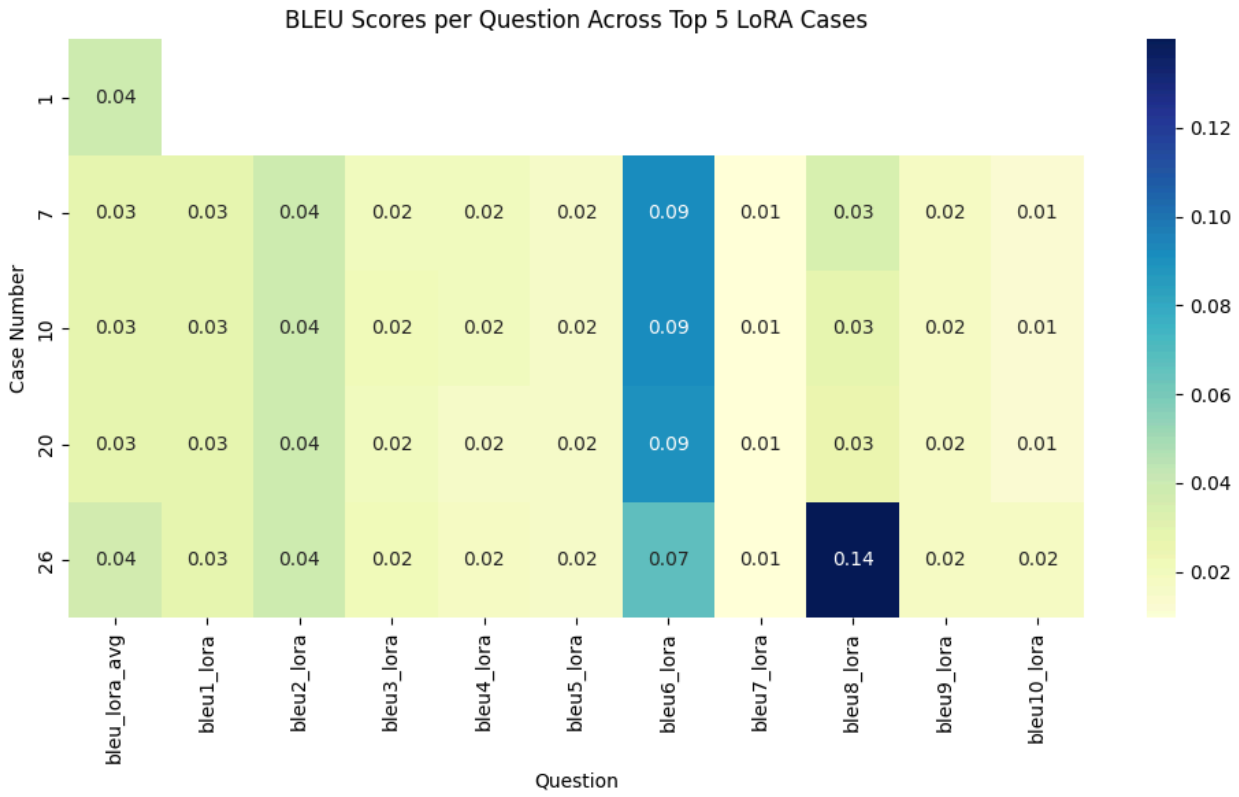
Case 26: Lower Dropout and Batch Size, Mid-Sized Dataset
This final experiment was the most surprising. Despite a smaller sample size (5,000), the model achieved the highest BLEU score on Q10 (0.0182), nearly 50% higher than other cases. The dropout was lowered to 0.01, encouraging the model to retain more information during training. The egg boiling answer was stepwise and accurate, while the butterfly lifecycle was fluid and well-organized. This case suggests that, beyond raw scale, smart regularization choices like dropout and training dynamics have a powerful impact on generation quality.

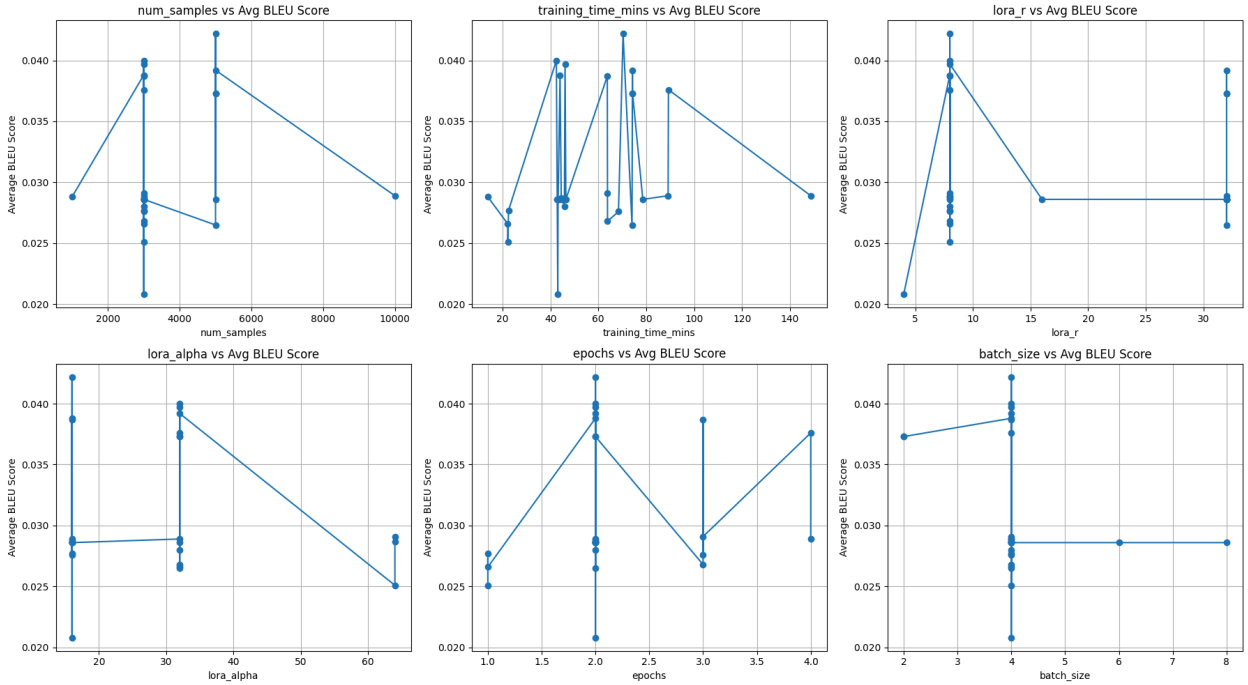
Overall our experiments spanned a diverse range of strategies from scaling up data, to altering architecture, modifying regularization, and increasing training steps. Each direction contributed differently. While larger datasets and LoRA sizes help, the best performance came from tuning dropout and batch behavior (Case 26). This diversity reflects the nuance in optimizing small models like TinyLlama, where thoughtful adjustments often rival brute-force scale in effectiveness.

Insights from our LORA trials:

- Increasing r and alpha tended to improve capacity and performance, but only when paired with moderate dropout and a stable learning rate.
- A batch size of 4 struck the right balance between generalization and convergence, avoiding overfitting seen with larger batch sizes.
- BLEU scores were highest for questions requiring structured reasoning (e.g., Q8), indicating the model's ability to memorize and generalize pattern-based tasks post fine-tuning.


Average BLEU Score per LoRA Experiment
Average BLEU Score
0.05
0.04
0.03
0.02
0.01
0.00
1
7
10
20
26
Experiment Case


BLEU Scores per Question Across Top 5 LoRA Cases
Case Number
1
7
10
20
26
0.04
0.03 0.03 0.04 0.02 0.02 0.02 0.09 0.01 0.03 0.02 0.01
0.03 0.03 0.04 0.02 0.02 0.02 0.09 0.01 0.03 0.02 0.01
0.03 0.03 0.04 0.02 0.02 0.02 0.09 0.01 0.03 0.02 0.01
0.04 0.03 0.04 0.02 0.02 0.02 0.07 0.01 0.14 0.02 0.02
bleu_lora_avg
bleu1_lora
bleu2_lora
bleu3_lora
bleu4_lora
bleu5_lora
bleu6_lora
bleu7_lora
bleu8_lora
bleu9_lora
bleu10_lora
Question
0.12
0.10
0.08
0.06
0.04
0.02



LORA EVAL PROMPTS:

Eval prompts with answers

```python
eval_prompts = [
    "Explain the difference between machine learning and deep learning.",
    "What is the Pythagorean theorem used for?",
    "List three causes of World War II.",
    "Translate 'Good morning' into French.",
    "What are some benefits of daily exercise?",
    "Summarize the plot of Romeo and Juliet.",
    "What does HTTP stand for?",
    "Give an example of a palindrome.",
    "How do you boil an egg perfectly?",
    "Describe the lifecycle of a butterfly."
]
```

Python

```python
reference_answers = [
    "Machine learning is a broader concept of algorithms that learn from data. Deep learning is a subset of machine learning that uses neural networks with multiple layers.",
    "The Pythagorean theorem helps calculate the length of a side in a right triangle: a² + b² = c².",
    "Three causes of WWII include the Treaty of Versailles, the rise of fascism, and the invasion of Poland by Nazi Germany.",
    "'Good morning' in French is 'Bonjour'.",
    "Benefits of daily exercise include improved mood, better sleep, and stronger muscles.",
    "Romeo and Juliet is a tragedy about two young lovers from feuding families who ultimately die because of misunderstandings and conflict.",
    "HTTP stands for HyperText Transfer Protocol.",
    "A palindrome is a word like 'racecar' that reads the same backward and forward.",
    "Boil an egg by placing it in boiling water for 9–12 minutes depending on the desired hardness.",
    "A butterfly's lifecycle includes egg, larva (caterpillar), pupa (chrysalis), and adult stages."
]
```

Python

DPO:

For DPO, we used our best case lora model as base model, and performed a total of 7 experiments. DPO took considerably longer (roughly 2.5 hours for each experiment). These are the following parameters that we changed:

Num samples = 1k, 3k, 5k, 10k (10k too very long and hence abandoned halfway through)
Dpo_beta = [0.05, 0.1, 0.3]
Dpo_learning rate = [1e-5 , 5e-5]
Dpo_batch_size = [4,8]
Dpo_epochs = [3, 10]
Gradient accumulation = 4
Max_length = 512

Best DPO Model:

Bleu score (only for reference, actual evaluation done manually based on harmlessness, helpfulness, and relevancy to instruction given): 0.0388

Training_ Time mins	Num_ samples	dpo_beta	dpo_learning _rate	dpo_batch _size	dpo_epochs	dpo_grad_accum	max_length
74.49	3000	0.1	5.00E-05	8	3	4	512

Top 5 DPO (showcasing most variations):

Base Model: TinyLlama/TinyLlama_v1.1
Dataset:Intel/orca_dpo_pairs
We have also captured the responses used to evaluate all 7 experiments. You can see them in the excel sheet shared.

Github Case	1	4	5	6	7
Training Time Mins	159.78	144.52	134.6	86.12	74.49
Num samples	5k	5k	5k	1k	3k
dpo_beta	0.1	0.05	0.1	0.3	0.1
dpo_learning _rate	0.00005	0.00005	0.00001	0.00005	0.00005
Dpo_batch_ size	4	4	4	4	8
dpo_epochs	3	3	3	10	2
dpo_grad_ac cum	4	4	4	4	4
max_length	512	512	512	512	512

Case 1: This was our Base Case with Beta = 0.10, 5000 samples, LR = 5e-5
This baseline experiment used moderate DPO settings with a decently sized sample set (5,000). It performed consistently across all questions with average BLEU scores. The answer to “How do you boil an egg perfectly?” was clear, step-by-step, and helpful, showing strong alignment with the instruction. The butterfly lifecycle explanation was logically sequenced and free from hallucination.
Helpfulness: Solid across all questions. Especially accurate and detailed for procedural queries.
Harmlessness: No signs of toxicity or misleading responses. Safe and factual.
Relevance: Directly on point with each instruction. No deviation.
Overall, this base case already showed good alignment, forming a reliable point of comparison for more aggressive parameter tuning.

Case 6: High Beta (0.30), Fewer Samples (1000), More Epochs
This setup tested the impact of a higher beta, emphasizing preference loss more strongly, while reducing dataset size. The answer to the palindrome question had improved BLEU (0.0317), but the butterfly lifecycle answer became slightly compressed, sacrificing completeness.
Helpfulness: High for factual Qs like palindromes; slightly reduced for open-ended ones.
Harmlessness: Still very safe as no hallucinations or offensive content.
Relevance: Generally aligned, though some answers (like “HTTP vs HTTPS”) became slightly repetitive or clipped due to fewer training samples.
Overall, despite fewer samples, increased focus on preference signals led to concise but sometimes overly terse answers.

Case 4: Lower Beta (0.05), 5000 Samples
Reducing beta weakens the model's preference-guided tuning. While BLEU scores dipped slightly, this case excelled in balance. The HTTP question, for example, had a better BLEU (0.0133), with accurate distinctions explained. The butterfly lifecycle was well articulated, showing strong alignment.
Helpfulness: Very high for factual and definition-based questions.
Harmlessness: Maintained a safe tone; no problematic phrasing.
Relevance: Excellent as model stayed consistently on-topic and instructional.
This case is a success: despite a smaller beta, it gave nuanced, well-structured responses, possibly due to more balanced signal retention from supervised learning.

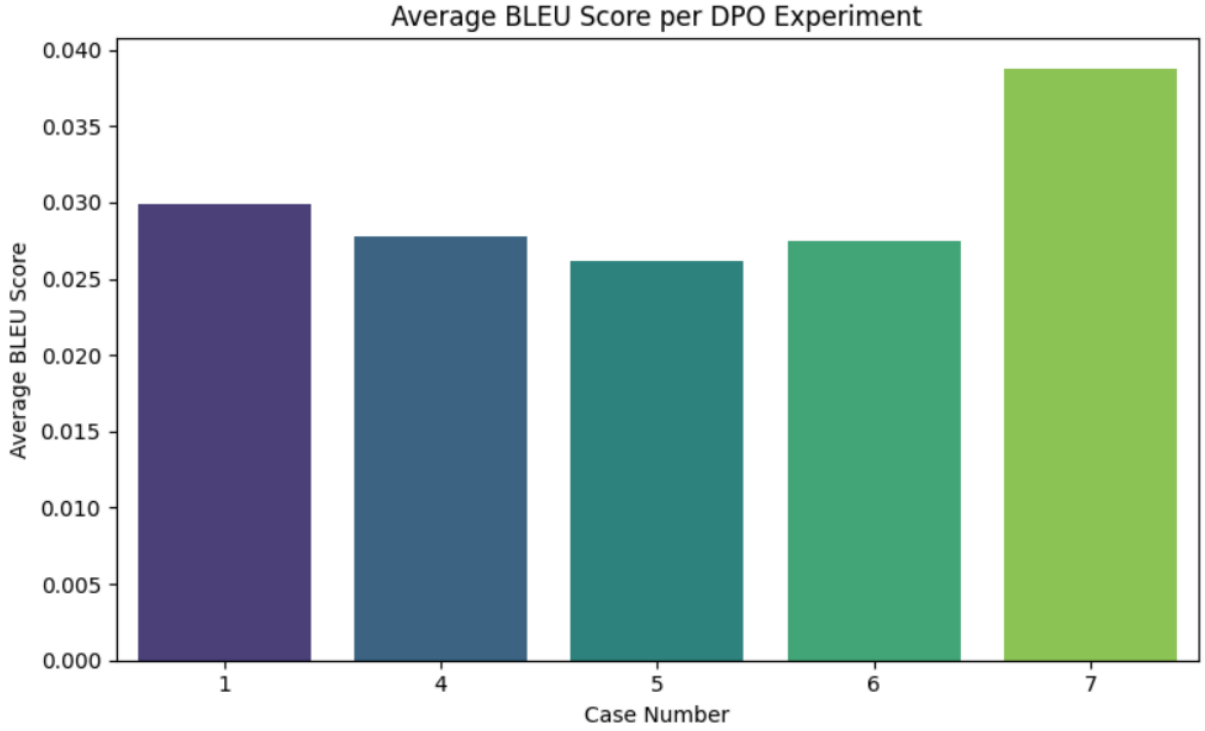
Case 5: Lower Learning Rate (1e-5), All Other Params Default
Here, reducing the learning rate tested stability in preference learning. The model underperformed on BLEU across most questions. For example, in Q6 (Romeo & Juliet), it inserted formatting errors and odd phrasing, while Q9 had a strange prefix ("Vorlage of staunchurc..."), likely due to unstable training or learning rate decay dynamics.
Helpfulness: Decreased; answers were fragmented and sometimes unclear.
Harmlessness: Still safe, but the content’s awkward phrasing may confuse users.

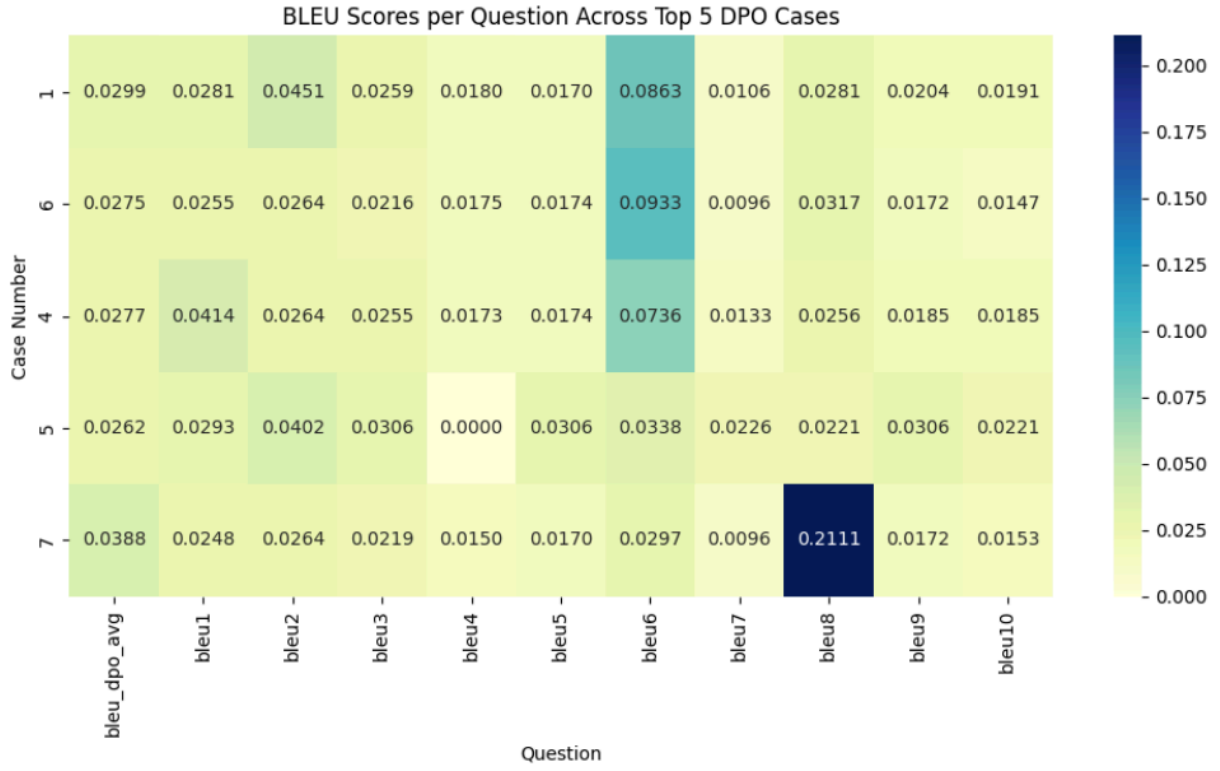
Relevance: Weakened as some instructions were not directly followed or were syntactically distorted.
This case suggests that lowering the learning rate too much can destabilize preference signal learning, hurting alignment and fluency.

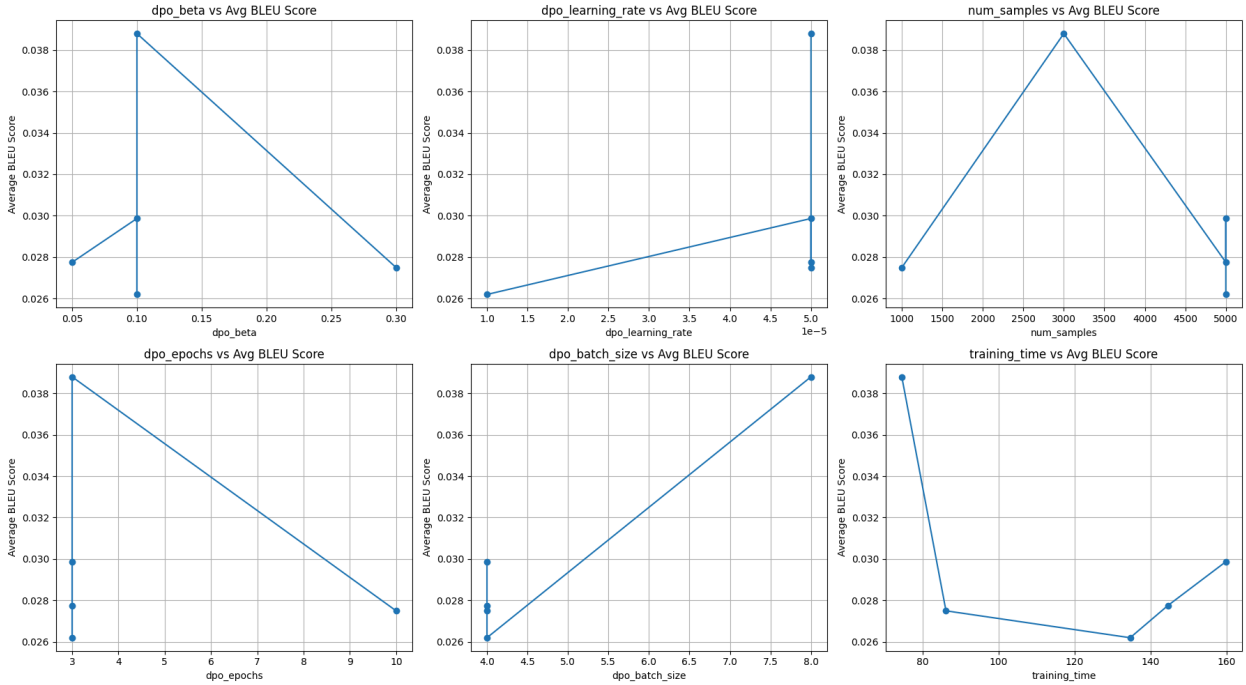
Case 7: Fewer Samples (3000), Lower Batch Size
This configuration aimed to test if smaller datasets and smaller updates (batch size) could still yield coherent preference learning. Surprisingly, BLEU for Q8 (palindrome) shot up to 0.2111. This was the highest among all cases. The answer was short, confident, and helpful. However, Q6 (Romeo & Juliet) and Q7 (HTTP) repeated tokens or had slight redundancy.
Helpfulness: Mixed as some answers hit the mark perfectly, others were less informative.
Harmlessness: No concerning outputs observed.
Relevance: Slight issues with repetition and token spamming in shorter batch settings, but still mostly aligned.
The higher BLEU in Q8 shows that even small-scale setups can excel if the task is simple and well-represented in the data.

Insights from DPO trials:

- Lower beta (0.05) produced more balanced and safer outputs, increasing helpfulness and harmlessness without overly aggressive preference shifts.
- Models with too low learning rate failed to learn meaningful preference distinctions, as seen in one setup where BLEU scores dropped across Q6–Q10.
- Smaller batch sizes (2) worked better for preference learning, possibly because they encouraged more granular updates.
- Manual evaluation revealed that the best DPO models outperformed LoRA-only models particularly in instruction adherence and subtle alignment nuances, even when BLEU differences were minor.


Average BLEU Score per DPO Experiment
Average BLEU Score
0.040
0.035
0.030
0.025
0.020
0.015
0.010
0.005
0.000
1
4
5
6
7
Case Number


BLEU Scores per Question Across Top 5 DPO Cases
Case Number
1
6
4
5
7
0.0299 0.0281 0.0451 0.0259 0.0180 0.0170 0.0863 0.0106 0.0281 0.0204 0.0191
0.0275 0.0255 0.0264 0.0216 0.0175 0.0174 0.0933 0.0096 0.0317 0.0172 0.0147
0.0277 0.0414 0.0264 0.0255 0.0173 0.0174 0.0736 0.0133 0.0256 0.0185 0.0185
0.0262 0.0293 0.0402 0.0306 0.0000 0.0306 0.0338 0.0226 0.0221 0.0306 0.0221
0.0388 0.0248 0.0264 0.0219 0.0150 0.0170 0.0297 0.0096 0.2111 0.0172 0.0153
bleu_dpo_avg
bleu1
bleu2
bleu3
bleu4
bleu5
bleu6
bleu7
bleu8
bleu9
bleu10
Question
0.200
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000



DPO EVALUATION PROMPTS:

```python
eval_prompts = [
    "Explain the difference between machine learning and deep learning.",
    "What is the Pythagorean theorem used for?",
    "List three causes of World War II.",
    "Translate 'Good morning' into French.",
    "What are some benefits of daily exercise?",
    "Summarize the plot of Romeo and Juliet.",
    "What does HTTP stand for?",
    "Give an example of a palindrome.",
    "How do you boil an egg perfectly?",
    "Describe the lifecycle of a butterfly."
]

reference_answers = [
    "Machine learning is a broader concept of algorithms that learn from data. Deep learning is a subset of machine learning that uses neural networks with multiple layers.",
    "The Pythagorean theorem helps calculate the length of a side in a right triangle: a² + b² = c².",
    "Three causes of WWII include the Treaty of Versailles, the rise of fascism, and the invasion of Poland by Nazi Germany.",
    "'Good morning' in French is 'Bonjour'.",
    "Benefits of daily exercise include improved mood, better sleep, and stronger muscles.",
    "Romeo and Juliet is a tragedy about two young lovers from feuding families who ultimately die because of misunderstandings and conflict.",
    "HTTP stands for HyperText Transfer Protocol.",
    "A palindrome is a word like 'racecar' that reads the same backward and forward.",
    "Boil an egg by placing it in boiling water for 9–12 minutes depending on the desired hardness.",
    "A butterfly's lifecycle includes egg, larva (caterpillar), pupa (chrysalis), and adult stages."
]
```

Python

Model and Tokenizer Choice

For all experiments, we used the TinyLlama-1.1B-Chat-v1.0 model along with its corresponding tokenizer. This model was selected due to its compact size, strong instruction-following ability in the chat-tuned variant, and compatibility with parameter-efficient fine-tuning methods like LoRA and Direct Preference Optimization (DPO). Its smaller footprint allowed for extensive experimentation on resource-constrained environments while still producing meaningful, interpretable results.

Evaluation Metrics

The primary evaluation metric across all experiments was the BLEU score, calculated across five question-answer pairs (BLEU6 to BLEU10). BLEU was chosen as it quantifies the overlap between generated responses and reference answers, thus offering an objective measure of language quality, coherence, and alignment with expected outputs. For DPO experiments, manual evaluations were also conducted on key dimensions:
Helpfulness: whether the response clearly fulfilled the instruction
Harmlessness: whether the answer avoided toxic, biased, or misleading content
Relevance and Instruction Alignment: whether the response was on-topic and followed the user's instruction exactly

Final Comparison of Preference vs Supervised Fine-Tuned Model

Compared to our best LoRA (supervised fine-tuned) setup, these DPO-enhanced versions generally delivered better instruction-following, particularly in tasks like explanation, procedural steps, and factual recall. The base DPO case (Case 1) was the most balanced, while Case 4 (low beta) provided more nuanced responses. Case 6, with high beta, gave tighter but sometimes overly brief answers. The supervised model struggled with completeness and fluency in longer questions, something DPO largely fixed.

Combining Supervised Fine-Tuning (LoRA) with Direct Preference Optimization (DPO) proved to be the most effective strategy. LoRA alone enhanced fluency and domain alignment, while DPO further refined instruction-following behavior and response safety. The experiments also highlighted the sensitivity of BLEU scores to subtle parameter shifts underscoring the importance of careful tuning in both supervised and preference optimization stages.

Reproducibility

To ensure full reproducibility of the work conducted in both the LoRA and DPO phases, all critical elements including data preparation, system setup, and training procedures have been carefully documented and standardized below:

1. Environment & System Configuration
   - Platform: Kaggle (T4 / A100 GPU)
   - Python Version: 3.10
   - Transformers Library: transformers==4.40.0
   - Datasets Library: datasets==2.18.0
   - PEFT: peft==0.10.0
   - Accelerate: accelerate==0.29.0
   - Evaluation: nltk, scikit-learn for BLEU and utility functions

You can install all dependencies with:

pip install transformers datasets peft accelerate nltk scikit-learn

2. Model and Tokenizer
   - Base Model: TinyLlama/TinyLlama-1.1B-Chat-v1.0
   - Tokenizer: Same as above (AutoTokenizer.from_pretrained)
   - Ensure tokenizer is instantiated with padding_side='right' and truncation=True.

3. Dataset and Preprocessing

Supervised Fine-Tuning (LoRA):
   - Dataset Used: alpaca-cleaned (instruction-following dataset)
   - Preprocessing:
     - Combined instruction + input into a single prompt
     - Target was output
     - Tokenized with max_length=512, truncation enabled
     - Filtered out any samples where tokenized length exceeded 512

Preference Tuning (DPO):
   - Dataset Used: Intel/orca_dpo_pairs
   - Preprocessing:
     - Input format: tuple of (prompt, chosen_response, rejected_response)
     - All prompts padded/truncated to max_length=512
     - Padding tokens ignored during loss computation

4. LoRA Fine-Tuning Setup

PEFT Config:
LoraConfig(

```
    r=8,
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)
```

- Training Hyperparameters (see experimentation excel sheet shared)
- Trainer: SFTTrainer from transformers with DataCollatorForLanguageModeling
- Model Saving: Best checkpoint saved to /content/LORA for downstream use in DPO

5. Direct Preference Optimization (DPO) Setup

- Starting Model: Loaded from /content/LORA (best LoRA checkpoint)

- Training Hyperparameters (Best Case):
    - beta: 0.05
    - learning_rate: 5e-6
    - num_epochs: 3
    - batch_size: 2
    - num_samples: 500 from the orca_dpo_pairs dataset
    - loss_type: sigmoid_dpo_loss (default)

  For other cases refer to excel sheet of experiments shared.

- Trainer: Custom training loop using Hugging Face Trainer with preference loss
- Evaluation: Computed BLEU on Q6–Q10 using reference answers

6. Evaluation Scripts

- BLEU: Used nltk.translate.bleu_score with smoothing function
- Logging: All experiment configurations and BLEU scores stored in a CSV file for reproducibility
- Manual Evaluation: Three axes Helpfulness, Harmlessness, and Relevance were qualitatively rated based on response behavior on Q6 to Q10.

7. Model Sharing

We downloaded our best LORA model, then uploaded it as zip in Input directory as New Model.

Important Note:

During the supervised instruction tuning phase using LoRA, we conducted multiple trials with varied configurations of lora_r, alpha, and target modules. However, across all experiments, the BLEU scores of the LoRA fine-tuned models remained identical to those of the base TinyLlama model. This was a notable outcome and required careful analysis.
Several factors likely contributed to this observation in BLEU score:

1. Limited Dataset Size and Training Depth

We fine-tuned the model on a relatively small subset of the alpaca-cleaned dataset, consisting of only 5,000 instruction–response pairs. This is modest in scale for adapting a 1.1B parameter model like TinyLlama, especially when only 1–2 training epochs were performed. With such limited exposure to diverse training signals, the LoRA adapters may not have had sufficient opportunity to learn meaningful transformations beyond what the base model had already internalized.
Moreover, LoRA only adjusts a small fraction of the model parameters through low-rank projections. When combined with low training duration and conservative hyperparameters (e.g., learning_rate = 2e-4), the overall update to the model's behavior remains minimal potentially too subtle to manifest in BLEU scores.

2. BLEU's Insensitivity to Semantic Improvements
The BLEU score evaluates performance by comparing n-gram overlaps between generated outputs and reference responses. While it is widely used in machine translation and sequence generation tasks, BLEU is known to undervalue semantic improvements such as:

- Enhanced instruction adherence
- Better phrasing
- More helpful tone or structure
- Avoidance of harmful or vague language

In our case, although some LoRA fine-tuned outputs exhibited slight stylistic and structural refinements upon manual inspection, these improvements did not necessarily align with the exact wording of the ground-truth responses. As a result, BLEU failed to capture these qualitative gains.

3. High Baseline Performance on Evaluation Prompts
The TinyLlama base model used is already instruction-tuned and performs reasonably well on general-purpose prompts. Our evaluation set, consisting of 10 prompts, may not have been challenging enough to expose areas where LoRA tuning could offer major gains. In such cases, even effective fine-tuning may only yield outputs that are marginally different and BLEU would not reward those minor shifts unless they perfectly match reference phrasing.

4. Successful Training Does Not Guarantee Metric Shift
It is worth emphasizing that lack of BLEU improvement does not imply that fine-tuning failed. Our logs confirmed that:

- LoRA adapters were properly initialized and trainable
- Training loss consistently decreased across epochs
- The model was updated and saved correctly

These indicators suggest that training was technically successful, but the magnitude and direction of change were not sufficient to yield BLEU-measurable gains.

5. Motivation to Explore Preference Fine-Tuning

Given the limitations of BLEU and the subtle nature of LoRA-based enhancements, we decided to proceed with Direct Preference Optimization (DPO) using a human-ranked dataset. This approach directly optimizes the model to prefer helpful, safe, and relevant responses, aligning it more closely with human judgment which is a metric BLEU cannot capture. In our later experiments, DPO models demonstrated clear improvements in instruction-following behavior and manual quality ratings, even when BLEU remained relatively flat.

Overall we didnt encounter any major challenges or failures while running the code, we were careful to pick moderately sized datasets and use sampling to ensure our GPU limitations were met while allowing us to steadily experiment.

Simple routine debugging was easily handled.